In [ ]:
```
import marimo as mo
import matplotlib.pyplot as plt
import torch
import torch.nn as nn

from torch.utils.data import DataLoader, Subset
from torchvision import datasets, transforms

import snntorch as snn
from snntorch import surrogate
```

Спайковая нейронная сеть для MNIST

Эксперимент по классификации рукописных цифр с помощью спайковой нейронной сети (SNN).

Архитектура

- Вход: 784 признака (изображение 28 × 28).
- Скрытый слой: 128 LIF-нейронов.
- Выходной слой: 10 LIF-нейронов.
- Временной горизонт: 10 шагов.
- Обучение: Adam и суррогатный градиент.
- Функция потерь: CrossEntropyLoss по суммарным спайкам.

Изображение подаётся на каждом шаге времени. Предсказание определяется по количеству спайков каждого выходного нейрона.

Используется уменьшенная выборка MNIST, чтобы ускорить эксперимент. Размеры выборок можно изменить ниже.

In [ ]:
```
device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

print("Устройство:", device)

if device.type == "cuda":
    print("GPU:", torch.cuda.get_device_name(device))
```

```
Устройство: cuda
GPU: NVIDIA GeForce RTX 3070 Ti Laptop GPU
```

In [ ]:
```
transform = transforms.ToTensor()

train_full = datasets.MNIST(
    root="./data",
    train=True,
    download=True,
```

```
    transform=transform,
)

test_full = datasets.MNIST(
    root="./data",
    train=False,
    download=True,
    transform=transform,
)

# Размеры подвыборок.
# Для более серьёзного эксперимента увеличь их.
TRAIN_SIZE = 12000
TEST_SIZE = 2000

train_set = Subset(
    train_full,
    range(min(TRAIN_SIZE, len(train_full))),
)

test_set = Subset(
    test_full,
    range(min(TEST_SIZE, len(test_full))),
)

train_loader = DataLoader(
    train_set,
    batch_size=128,
    shuffle=True,
    num_workers=0,
    pin_memory=False,
)

test_loader = DataLoader(
    test_set,
    batch_size=256,
    shuffle=False,
    num_workers=0,
    pin_memory=False,
)

print("Обучающих изображений:", len(train_set))
print("Тестовых изображений:", len(test_set))
```

```
Обучающих изображений: 12000
Тестовых изображений: 2000
```

In [ ]:

```
class LIFNet(nn.Module):
    def __init__(
        self,
        num_steps=10,
        hidden=128,
        beta=0.9,
    ):
        super().__init__()
```

```
        self.num_steps = num_steps
        self.hidden = hidden

        self.fc1 = nn.Linear(784, hidden)

        self.lif1 = snn.Leaky(
            beta=beta,
            spike_grad=surrogate.fast_sigmoid(
                slope=25
            ),
        )

        self.fc2 = nn.Linear(hidden, 10)

        self.lif2 = snn.Leaky(
            beta=beta,
            spike_grad=surrogate.fast_sigmoid(
                slope=25
            ),
        )

    def forward(self, images):
        # images: [batch, 1, 28, 28]
        x = images.reshape(images.shape[0], -1)

        batch_size = x.shape[0]

        # Начальные мембранные потенциалы.
        # Состояния создаются заново при каждом forward,
        # поэтому информация между изображениями
        # не переносится.
        mem1 = x.new_zeros(
            batch_size,
            self.hidden,
        )

        mem2 = x.new_zeros(
            batch_size,
            10,
        )

        spike_record = []

        for _ in range(self.num_steps):
            # Входной ток скрытого слоя.
            current1 = self.fc1(x)

            # Скрытые LIF-нейроны.
            spike1, mem1 = self.lif1(
                current1,
                mem1,
            )

            # Входной ток выходного слоя.
            current2 = self.fc2(spike1)
```

```
            # Выходные LIF-нейроны.
            spike2, mem2 = self.lif2(
                current2,
                mem2,
            )

            spike_record.append(spike2)

        # [time, batch, classes]
        return torch.stack(
            spike_record,
            dim=0,
        )
```

In [ ]:
```
model = LIFNet(
    num_steps=10,
    hidden=128,
    beta=0.9,
).to(device)

print(model)

total_params = sum(
    parameter.numel()
    for parameter in model.parameters()
    if parameter.requires_grad
)

print("Обучаемых параметров:", total_params)
```

```
LIFNet(
  (fc1): Linear(in_features=784, out_features=128, bias=True)
  (lif1): Leaky()
  (fc2): Linear(in_features=128, out_features=10, bias=True)
  (lif2): Leaky()
)
Обучаемых параметров: 101770
```

In [ ]:
```
criterion = nn.CrossEntropyLoss()
```

In [ ]:
```
optimizer = torch.optim.Adam(
    model.parameters(),
    lr=0.001,
)
```

In [ ]:
```
def train_one_epoch():
    model.train()

    total_loss = 0.0
    total_correct = 0
    total_seen = 0

    current_device = next(
        model.parameters()
    ).device
```

```
    for images, labels in train_loader:
        images = images.to(current_device)
        labels = labels.to(current_device)

        optimizer.zero_grad(set_to_none=True)

        # [time, batch, classes]
        spikes = model(images)

        # Число спайков каждого выходного нейрона:
        # [batch, classes]
        spike_counts = spikes.sum(dim=0)

        loss = criterion(
            spike_counts,
            labels,
        )

        loss.backward()

        optimizer.step()

        batch_size = labels.size(0)

        total_loss += (
            loss.item() * batch_size
        )

        predictions = spike_counts.argmax(dim=1)

        total_correct += (
            predictions == labels
        ).sum().item()

        total_seen += batch_size

    mean_loss = total_loss / max(total_seen, 1)

    accuracy = (
        total_correct / max(total_seen, 1)
    )

    return mean_loss, accuracy
```

In [ ]:

```
@torch.no_grad()
def evaluate():
    model.eval()

    current_device = next(
        model.parameters()
    ).device

    total_correct = 0
    total_seen = 0

    for images, labels in test_loader:
```

```
        images = images.to(current_device)
        labels = labels.to(current_device)

        spikes = model(images)

        spike_counts = spikes.sum(dim=0)

        predictions = spike_counts.argmax(dim=1)

        total_correct += (
            predictions == labels
        ).sum().item()

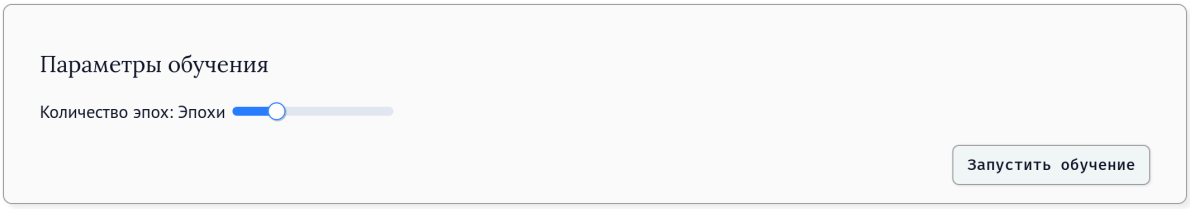
        total_seen += labels.size(0)

    return (
        total_correct / max(total_seen, 1)
    )
```

In [ ]:
```
training_form = mo.ui.form(
    mo.md("""
    ### Параметры обучения

    Количество эпох: {epochs}
    """).batch(
        epochs=mo.ui.slider(
            start=1,
            stop=5,
            value=2,
            step=1,
            label="Эпохи",
        )
    ),
    submit_button_label="Запустить обучение",
)

training_form
```



In [ ]:
```
# До отправки формы обучение не запускается.
if training_form.value is None:
    mo.md("""
    Выбери количество эпох и нажми
    **«Запустить обучение»**.
    """)

else:
    num_epochs = int(
        training_form.value["epochs"]
```

```
    )

    results = []

    for epoch_idx in range(1, num_epochs + 1):
        loss_value, train_accuracy = (
            train_one_epoch()
        )

        test_accuracy = evaluate()

        row = {
            "Эпоха": epoch_idx,
            "Loss": round(loss_value, 4),
            "Обучение (%)": round(
                train_accuracy * 100, 2
            ),
            "Тест (%)": round(
                test_accuracy * 100, 2
            ),
        }

        results.append(row)

        print(
            f"Эпоха {epoch_idx}/{num_epochs} | "
            f"Loss: {loss_value:.4f} | "
            f"Обучение: "
            f"{train_accuracy * 100:.2f}% | "
            f"Тест: {test_accuracy * 100:.2f}%"
        )

    mo.md("### Результаты обучения")

    mo.ui.table(results)
```

```
Эпоха 1/2 | Loss: 0.7847 | Обучение: 74.55% | Тест: 88.00%
Эпоха 2/2 | Loss: 0.2497 | Обучение: 92.75% | Тест: 90.05%
```

In [ ]:
```
show_examples = mo.ui.run_button(
    label="Показать примеры распознавания"
)

show_examples
```

Показать примеры распознавания

In [ ]:
```
if show_examples.value:
    model.eval()

    current_device = next(
        model.parameters()
    ).device

    # Получаем изображения тестовой выборки.
    images, labels = next(iter(test_loader))
```

```
images = images[:12].to(current_device)
labels = labels[:12].to(current_device)

with torch.no_grad():
    spikes = model(images)

    # [batch, classes]
    spike_counts = spikes.sum(dim=0)

    predictions = spike_counts.argmax(dim=1)

# Переносим данные на CPU для визуализации.
images_cpu = images.cpu()
labels_cpu = labels.cpu()
predictions_cpu = predictions.cpu()
counts_cpu = spike_counts.cpu()

# ------------------------------------------------------
# 14.1. Изображения и предсказания
# ------------------------------------------------------

fig_examples, axes_examples = plt.subplots(
    3,
    4,
    figsize=(10, 8),
)

axes_flat = axes_examples.ravel()

for idx in range(len(images_cpu)):
    ax = axes_flat[idx]

    ax.imshow(
        images_cpu[idx, 0].numpy(),
        cmap="gray",
    )

    true_digit = int(labels_cpu[idx])
    predicted_digit = int(
        predictions_cpu[idx]
    )

    is_correct = (
        true_digit == predicted_digit
    )

    status = (
        "Верно" if is_correct else "Ошибка"
    )

    title_color = (
        "green" if is_correct else "red"
    )

    ax.set_title(
```

```
            f"Сеть: {predicted_digit}\n"
            f"Ответ: {true_digit}\n"
            f"{status}",
            color=title_color,
            fontsize=10,
        )

        ax.axis("off")

    fig_examples.tight_layout()
    plt.show()
    plt.close(fig_examples)

    # ------------------------------------------------------
    # 14.2. Таблица предсказаний
    # ------------------------------------------------------

    result_rows = []

    for idx in range(len(images_cpu)):
        true_digit = int(labels_cpu[idx])
        predicted_digit = int(
            predictions_cpu[idx]
        )

        result_rows.append({
            "№": idx + 1,
            "Правильная цифра": true_digit,
            "Предсказание": predicted_digit,
            "Верно": (
                true_digit == predicted_digit
            ),
        })

    mo.ui.table(result_rows)

    # ------------------------------------------------------
    # 14.3. Диаграмма спайков первого изображения
    # ------------------------------------------------------

    first_counts = (
        counts_cpu[0].numpy()
    )

    fig_counts, ax_counts = plt.subplots(
        figsize=(8, 3),
    )

    ax_counts.bar(
        range(10),
        first_counts,
    )

    ax_counts.set_xticks(range(10))
    ax_counts.set_xlabel("Класс цифры")
    ax_counts.set_ylabel(
```

```
        "Суммарное число спайков"
    )

    first_prediction = int(
        predictions_cpu[0]
    )

    first_answer = int(
        labels_cpu[0]
    )

    ax_counts.set_title(
        f"Предсказание: {first_prediction}; "
        f"правильный ответ: {first_answer}"
    )

    fig_counts.tight_layout()
    plt.show()
    plt.close(fig_counts)

    # ------------------------------------------------------
    # 14.4. Общая точность на показанных изображениях
    # ------------------------------------------------------

    correct_count = (
        predictions_cpu == labels_cpu
    ).sum().item()

    sample_accuracy = (
        correct_count / len(labels_cpu) * 100
    )

    mo.md(
        f"""
        ### Результат на выбранных изображениях

        - Правильно распознано: **{correct_count} из {len(labels_cpu)}**
        - Точность на этих изображениях: **{sample_accuracy:.2f}%**

        Это точность только на показанных изображениях,
        а не на всей тестовой выборке.
        """
    )

else:
    mo.md("""
    Нажми **«Показать примеры распознавания»**,
    чтобы посмотреть предсказания сети.
    """)
```


Сеть: 7
Ответ: 7
Верно
Сеть: 2
Ответ: 2
Верно
Сеть: 1
Ответ: 1
Верно
Сеть: 0
Ответ: 0
Верно
Сеть: 4
Ответ: 4
Верно
Сеть: 1
Ответ: 1
Верно
Сеть: 4
Ответ: 4
Верно
Сеть: 9
Ответ: 9
Верно
Сеть: 6
Ответ: 5
Ошибка
Сеть: 9
Ответ: 9
Верно
Сеть: 0
Ответ: 0
Верно
Сеть: 6
Ответ: 6
Верно


Предсказание: 7; правильный ответ: 7
Суммарное число спайков
0
2
4
6
8
10
0
1
2
3
4
5
6
7
8
9
Класс цифры